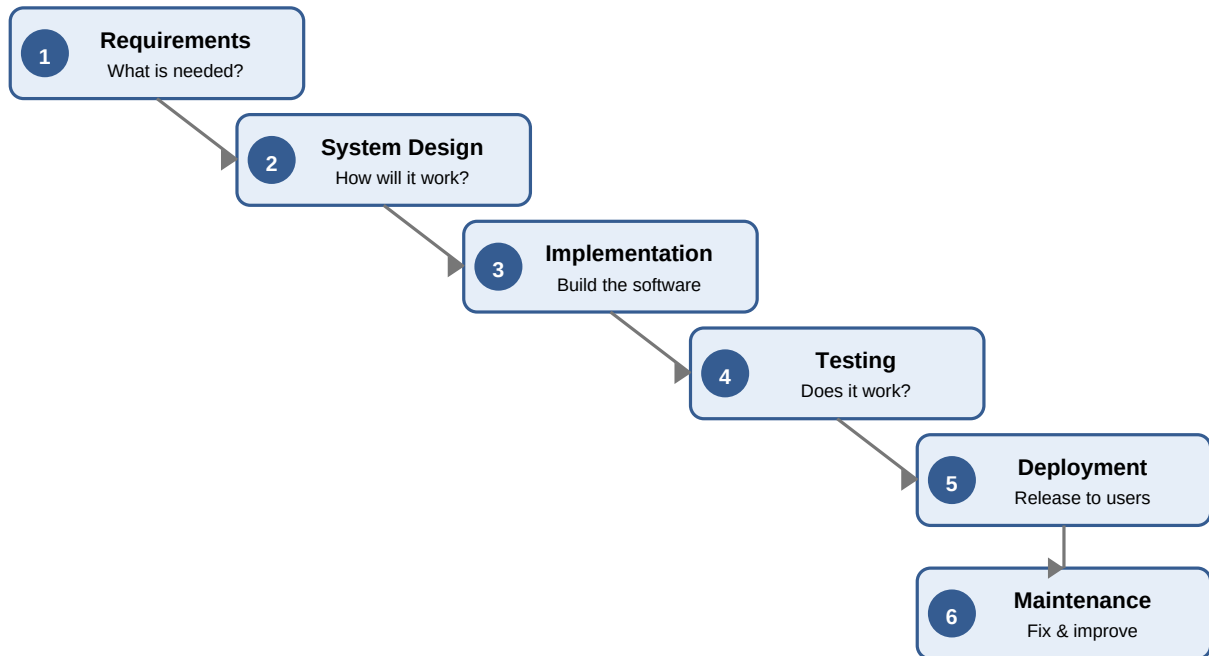


Waterfall Model in Software Engineering

A complete, student-friendly guide with diagrams, phase explanations, advantages, limitations, and a real-life project example

Waterfall Model



A sequential process: complete a phase, review its output, then move forward.

Figure 1. The basic sequential flow of the Waterfall Model.

1. What is the Waterfall Model?

The Waterfall Model is a traditional software development process model in which development activities are organized into a sequence of phases. The basic idea is simple: the team completes and documents one major phase before moving to the next. The model emphasizes upfront planning, clear requirements, documentation, formal reviews, and predictable progression.

It is called **Waterfall** because the process is often drawn like water flowing downward from one level to the next. Once the project moves forward, returning to an earlier phase can require rework. Therefore, the model works best when requirements are well understood and relatively stable.

Easy definition for students: Waterfall is a sequential SDLC model where software development moves phase by phase from requirements to design, coding, testing, deployment, and maintenance.

2. Basic Phases

A common practical representation contains six major phases: Requirements, System Design, Implementation, Testing, Deployment, and Maintenance. Some textbooks divide these phases into more detailed activities, such as feasibility study, architectural design, integration testing, and acceptance testing. The names can vary slightly, but the underlying sequential idea remains.

Phase	Simple Question	Main Result
Requirements	What does the customer need?	SRS / approved requirements
Design	How will the system work?	Architecture and detailed design
Implementation	How will we build it?	Source code / software

Phase	Simple Question	Main Result
Testing	Does it work correctly?	Test results and defect reports
Deployment	How will users get it?	Production release
Maintenance	How will we support it?	Fixes, updates, enhancements

3. Requirements Analysis

Purpose: The first major development activity is to understand what the customer and users actually need. The team collects requirements from stakeholders, analyzes them, removes ambiguity, identifies constraints, prioritizes requirements, and documents the agreed scope.

What happens: **Functional requirements** describe what the system should do, such as registering a user, placing an order, or generating a report. **Non-functional requirements** describe quality attributes and constraints, such as security, response time, availability, scalability, and usability.

Real-life FoodFast example: **FoodFast example:** The client specifies that customers must register, log in, search restaurants, view menus, add items to a cart, place orders, pay online, receive notifications, cancel eligible orders, and track delivery. The team also specifies security and performance requirements.

Typical deliverables: Software Requirements Specification (SRS), use cases, requirement list, acceptance criteria, scope statement, and requirement traceability information.

Student takeaway: If the team does not understand what the customer wants, building the right software becomes difficult.

4. System and Software Design

Purpose: The design phase converts approved requirements into a technical plan. The team decides how the system will be structured and how its components will communicate.

What happens: Design may include system architecture, module decomposition, database design, user-interface design, API design, security design, data flow, technology selection, and detailed component specifications. UML diagrams, ER diagrams, flowcharts, and prototypes may be used.

Real-life FoodFast example: **FoodFast example:** The team designs a mobile/web frontend, backend services, authentication, restaurant and menu modules, cart and order processing, payment integration, delivery tracking, notifications, and a database containing User, Restaurant, Menu, Order, Payment, and Delivery data.

Typical deliverables: Architecture diagram, UML diagrams, ER/database design, API specification, UI design, component design, and security design.

Student takeaway: Design is the blueprint. Developers should not begin complex implementation without understanding the intended structure.

5. Implementation / Coding

Purpose: In this phase, developers translate the approved design into source code. The system is usually divided into manageable modules so developers can implement and integrate them systematically.

What happens: Developers write code, create database scripts, configure dependencies, use version control, follow coding standards, perform code reviews, and integrate components. Unit-level checks are normally performed during development even though full system testing occurs later.

Real-life FoodFast example: **FoodFast example:** Developers create the login system, restaurant listing, menu display, cart, order-processing logic, payment integration, notifications, and delivery-tracking functionality. Frontend components communicate with backend services through APIs.

Typical deliverables: Source code, builds, modules, database scripts, configuration files, unit-test results, and developer documentation.

Student takeaway: Coding is not the entire software engineering process; it is one phase of a larger lifecycle.

6. Testing

Purpose: Testing checks whether the implemented software behaves correctly and satisfies the documented requirements. The goal is to discover defects before and around release and provide evidence of quality.

What happens: Testing can include unit testing, integration testing, system testing, regression testing, performance testing, security testing, usability testing, and user acceptance testing. Defects are logged, corrected, retested, and tracked.

Real-life FoodFast example: **FoodFast example:** Testers verify that cart totals are correct, successful payments create orders, failed payments do not create duplicate orders, cancellation rules work, delivery status is accurate, and the system

remains responsive when many customers use it simultaneously.

Typical deliverables: Test plan, test cases, test data, defect reports, test execution results, regression results, and release recommendation.

Student takeaway: Testing is not just 'finding bugs'; it is systematic verification against expected behavior and requirements.

7. Deployment / Release

Purpose: After required testing and approval, the software is moved from the development or test environment to the production environment where real users can access it.

What happens: Deployment may involve server configuration, database migration, environment variables, security settings, monitoring, backups, deployment scripts, documentation, and release management. A system can be released all at once or through a controlled rollout.

Real-life FoodFast example: The production backend is deployed to cloud infrastructure, the production database and payment gateway are configured, monitoring is enabled, and the mobile application is released. The company may initially launch in one city and expand later.

Typical deliverables: Production build, deployment configuration, release notes, operational documentation, monitoring setup, backup/rollback procedures.

Student takeaway: Deployment changes software from a tested project artifact into a service used by real people.

8. Maintenance

Purpose: Maintenance begins after deployment and may continue for years. The team keeps the software secure, reliable, compatible, and useful as users, technologies, and business needs change.

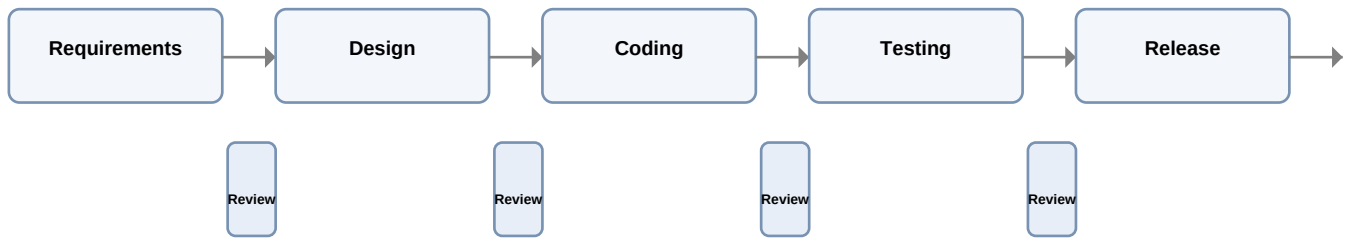
What happens: Maintenance can be corrective, adaptive, perfective, or preventive. Corrective maintenance fixes defects; adaptive maintenance responds to changes in operating environments; perfective maintenance improves or extends the system; preventive maintenance reduces the chance of future failures.

Real-life FoodFast example: The team fixes a payment bug, patches a security vulnerability, updates the application for a new mobile operating-system version, improves delivery tracking, optimizes database queries, and adds a coupon feature.

Typical deliverables: Patches, updates, new releases, performance improvements, security fixes, change records, and maintenance documentation.

Student takeaway: Software is not finished simply because it has been deployed.

Phase Reviews / Milestones



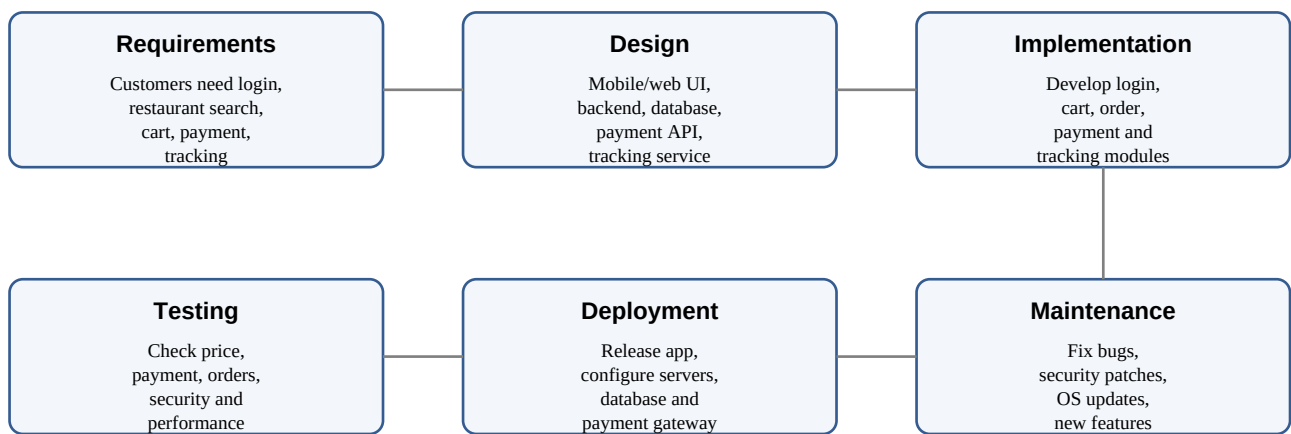
In practice, review and approval points help control quality and reduce uncontrolled changes.

Figure 2. Phase transitions can include reviews, approvals, and milestone checks.

9. Real-Life Example — FoodFast from Start to Finish

Suppose a company wants to build FoodFast because customers currently have to call restaurants or use several disconnected services. Management approves the project after a feasibility study. The team then freezes an agreed first version of the requirements and begins the sequential process.

Real-Life Example: FoodFast Online Food Delivery



If a major requirement changes late, earlier work may need to be revisited.

Figure 3. FoodFast mapped to the complete Waterfall sequence.

Step-by-step story: First, the team decides what FoodFast must do. Next, architects and designers decide how the application will work. Developers then implement the approved design. Testers verify the integrated system. Once the release is approved, the application goes live. Finally, the team maintains it by fixing defects and making controlled improvements.

10. Advantages of the Waterfall Model

Advantage	Why it helps
Simple structure	Students and project teams can easily understand the sequence.
Clear milestones	Each phase has identifiable goals and deliverables.
Strong documentation	Documents support review, handover, maintenance, and traceability.
Predictable planning	Stable requirements make cost and schedule estimation easier.
Defined responsibilities	Teams can organize work around clear phase activities.
Useful for stable projects	Projects with well-understood requirements can benefit from a structured process.

11. Disadvantages / Limitations

Limitation	Why it can be a problem
Late feedback	Users may not see a complete working product until relatively late.
Change is expensive	Major changes can require revisiting completed requirements, design, code, and tests.
Late discovery of misunderstandings	A requirement problem may become visible only when the system is tested or demonstrated.
Less flexible	Frequent changes and experimentation do not fit the model naturally.
Risk of over-documentation	Teams may spend substantial effort producing and maintaining documents.

12. When Should We Use Waterfall?

Waterfall is generally more appropriate when requirements are well understood, the scope is relatively stable, the technology is familiar, extensive documentation is valuable or required, and predictable planning is important. Examples can include some government, defense, regulated, contract-driven, or embedded-software projects where requirements and verification activities are defined in advance.

Important: Waterfall is not automatically 'bad' or 'old-fashioned.' The right development model depends on project characteristics. A model should be selected based on requirements, risk, regulation, customer involvement, technology uncertainty, and expected change.

13. When is Waterfall NOT a Good Choice?

Waterfall can be a poor fit when requirements are unclear, customers need frequent changes, the product is highly experimental, early user feedback is essential, or the technology is changing rapidly. In such situations, iterative or incremental approaches may provide earlier feedback and make change easier to manage.

14. Waterfall vs Agile — Quick Comparison

Aspect	Waterfall	Agile
Flow	Mostly sequential	Iterative and incremental
Requirements	Preferably stable early	Can evolve through iterations
Customer feedback	Often concentrated at defined milestones	Frequent throughout development
Delivery	Usually later / after major phases	Frequent working increments
Change	Can be costly after phase completion	Expected and managed through iterations
Planning	Detailed upfront planning is common	Planning is repeated at multiple levels
Best fit	Stable, predictable projects	Changing requirements and rapid feedback

15. Important Terms Students Should Know

Term	Student-friendly meaning
SRS	Software Requirements Specification; the documented statement of what the software should do.
Functional Requirement	A required system behavior or service.
Non-functional Requirement	A quality or constraint such as security, speed, availability, or usability.
Baseline	An approved version of requirements or design used as a controlled reference.
Milestone	A significant project checkpoint or achievement.
Verification	Checking whether work products are being built according to specified requirements and standards.
Validation	Checking whether the resulting system satisfies the user's intended needs.
Defect	A problem or deviation from expected behavior.
Regression Testing	Testing performed after changes to make sure existing functionality still works.
Deployment	Putting the software into a production environment for users.
Maintenance	Changes made after release to fix, adapt, improve, or prevent problems.

16. Common Exam / Viva Questions

1. What is the Waterfall Model?
2. Why is it called a Waterfall Model?
3. List the major phases of the Waterfall Model.
4. What is the purpose of the Requirements phase?
5. What is an SRS?
6. Why is the Design phase important?
7. What happens during Implementation?
8. Why is testing performed after implementation in the traditional Waterfall sequence?
9. What is deployment?
10. What is software maintenance?
11. Give two advantages of the Waterfall Model.
12. Give two disadvantages of the Waterfall Model.
13. Why can requirement changes be expensive in Waterfall?
14. What type of project is suitable for Waterfall?
15. How is Waterfall different from Agile?

17. Group Class Work

Group size: 4–5 students. Choose one project and prepare a Waterfall-based development plan.

Project choices: Student Attendance Management System; Hospital Management System; Online Food Ordering System; Library Management System; College Event Management System; Online Examination System; Hostel Management System; Placement Management System; E-learning Management System; Bus/Transport Management System.

Your group must prepare:

1. Write a short problem statement.
2. Identify the main stakeholders.
3. Write at least 8 functional requirements.

4. Write at least 4 non-functional requirements.
5. Draw the complete Waterfall diagram for your project.
6. Explain what happens in every Waterfall phase.
7. Give one realistic example for every phase.
8. List at least two deliverables for every phase.
9. Identify two risks caused by a requirement change after the Design phase.
10. Explain two advantages and two disadvantages of using Waterfall for your project.
11. Decide whether Waterfall is suitable for your project and justify your decision.
12. Prepare a 5–7 minute presentation. Every group member must participate.

18. Mini Case Study for Discussion

Situation: The FoodFast team has completed Requirements and Design. Developers have already started coding. The client suddenly says that the application must now support a completely different payment provider and a new payment workflow.

Discuss: Which earlier documents may need to change? Which design components may need rework? What code might be affected? Which test cases may need to be rewritten? What additional time and cost could result? Would an iterative model make this type of change easier to handle?

19. Quick Revision

Remember this sequence: Requirements → Design → Implementation → Testing → Deployment → Maintenance.

One-line memory trick: *Know it → Design it → Build it → Test it → Release it → Maintain it.*